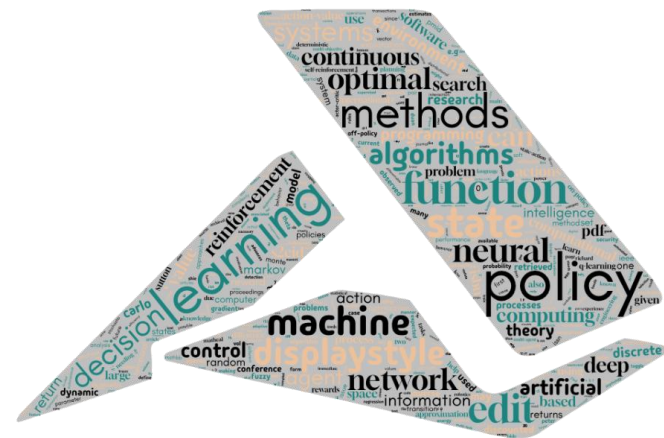
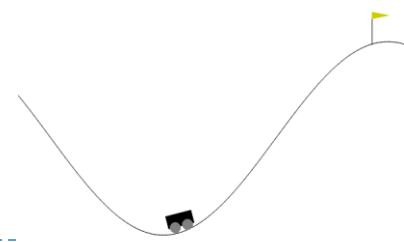




# Reinforcement Learning Algorithms and Applications

Chapter 11. Twin Delayed DDPG for  
Mountain Car Continuous

# Mountain Car Continuous



- ▶ **Mountain Car Continuous** is a classic **reinforcement learning** environment from the **Gymnasium** library.
- ▶ The agent controls a car stuck in a **valley** between two **hills** and must reach the **flag** at the top of the right hill.
- ▶ The car's **engine** is not strong enough to drive directly up the steep hill, requiring a **momentum-building strategy**.
- ▶ The agent must learn to **oscillate** back and forth to build up enough **kinetic energy** to reach the goal.
- ▶ **State space** consists of two continuous values: **position** (ranging from -1.2 to 0.6) and **velocity** (ranging from -0.07 to 0.07).
- ▶ **Action space** is a single continuous value representing the **force** applied to the car (ranging from -1.0 to 1.0).
- ▶ The goal is to reach a **position**  $\geq 0.45$ , which represents the **flag location** at the top of the right hill.
- ▶ This environment is particularly challenging because it requires **long-term planning** and **strategic exploration** rather than greedy actions.
- ▶ The **physics simulation** makes this problem ideal for testing **continuous control** algorithms like **DDPG**.

# TD3 vs DDPG: Key Differences

## ▶ Twin Critics (Reduce Overestimation)

- ▶ **DDPG:** Uses a single Q-network
- ▶ **TD3:** Uses two Q-networks ( $Q_1, Q_2$ )
  - ▶ Target Q-value:
  - ▶  $y = r + \gamma \min(Q_{target\_1}(s', a'), Q_{target\_2}(s', a'))$
  - ▶ Critic loss:
  - ▶ 
$$L_{critic} = \frac{1}{N} \sum_{i=1}^N [(Q_1(s_i, a_i) - y_i)^2 + (Q_2(s_i, a_i) - y_i)^2]$$

## ▶ Target Policy Smoothing

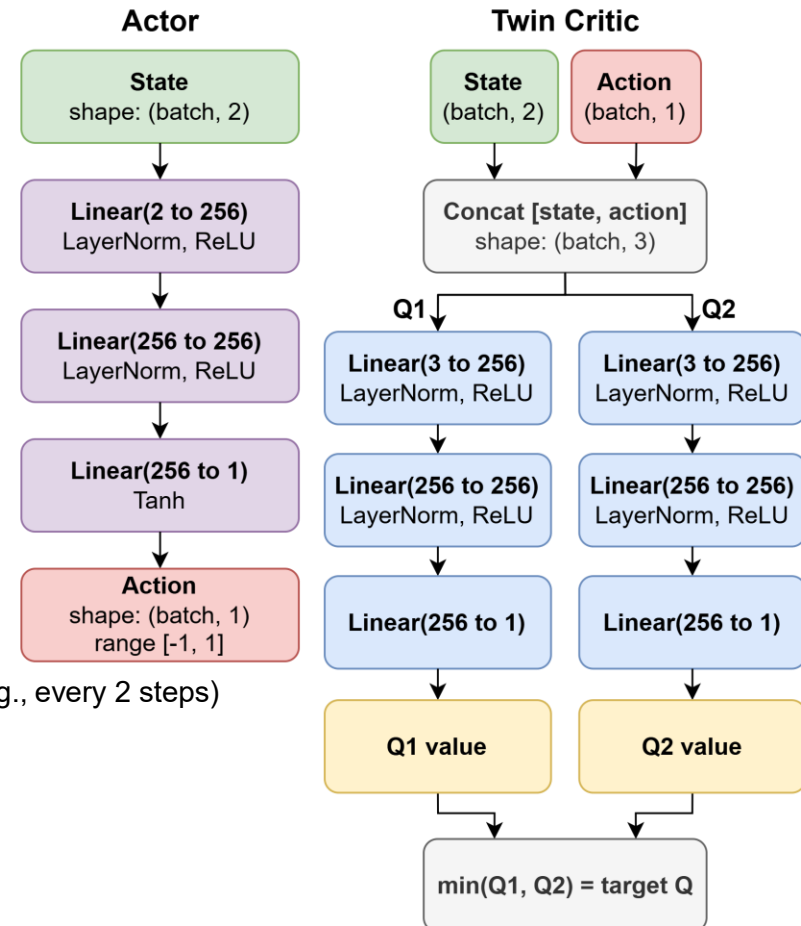
- ▶ **DDPG:** Uses deterministic target action
- ▶ **TD3:** Adds clipped noise to target action
  - ▶  $a' = \text{clip}(\mu_{target}(s') + \text{clip}(\epsilon, -c, c), a_{\min}, a_{\max}), \epsilon \sim \mathcal{N}(0, \sigma)$
  - ▶ Prevents exploitation of sharp Q-function peaks

## ▶ Delayed Policy Update

- ▶ **DDPG:** Updates Actor and Critic every step
- ▶ **TD3:** Updates  $\mu, \mu_{target}, Q_{target\_1}$ , and  $Q_{target\_2}$  less frequently (e.g., every 2 steps)
- ▶ Allows Critic to learn more accurately before guiding Actor

## ▶ Actor Update (Only One Q)

- ▶ TD3 uses only one Q-network ( $Q_1$ ) to update the Actor:
- ▶ Using  $\min(Q_1, Q_2)$  is **not suitable** for policy gradients:
  - ▶ Non-smooth  $\rightarrow$  unstable gradients
  - ▶ Higher variance in updates
- ▶ **One Q-network** is sufficient to provide the **gradient direction**
  - ▶ 
$$L_{actor} = -\frac{1}{N} \sum_{i=1}^N Q_1(s_i, \mu(s_i))$$



# DDPG and TD3 Training Algorithm

## Algorithm 1 Deep Deterministic Policy Gradient

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta$ ,  $\phi_{\text{targ}} \leftarrow \phi$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for however many updates do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute targets

```

$$y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$$

```

13:   Update Q-function by one step of gradient descent using

```

$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s, a, r, s', d) \in B} (Q_{\phi}(s, a) - y(r, s', d))^2$$

```

14:   Update policy by one step of gradient ascent using

```

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi}(s, \mu_{\theta}(s))$$

```

15:   Update target networks with

```

$$\begin{aligned} \phi_{\text{targ}} &\leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned}$$

```

16:   end for
17: end if
18: until convergence

```

## Algorithm 1 Twin Delayed DDPG

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta$ ,  $\phi_{\text{targ},1} \leftarrow \phi_1$ ,  $\phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for  $j$  in range(however many updates) do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute target actions

```

Adds clipped noise to target action

$$a'(s') = \text{clip}(\mu_{\theta_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

```

13:   Compute targets

```

Twin Critics (Reduce Overestimation)

$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', a'(s'))$$

```

14:   Update Q-functions by one step of gradient descent using

```

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s, a, r, s', d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

```

15:   if  $j \bmod \text{policy\_delay} = 0$  then

```

```

16:     Update policy by one step of gradient ascent using

```

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi_1}(s, \mu_{\theta}(s))$$

```

17:     Update target networks with

```

$$\begin{aligned} \phi_{\text{targ},i} &\leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned} \quad \text{for } i = 1, 2$$

Delayed Update

```

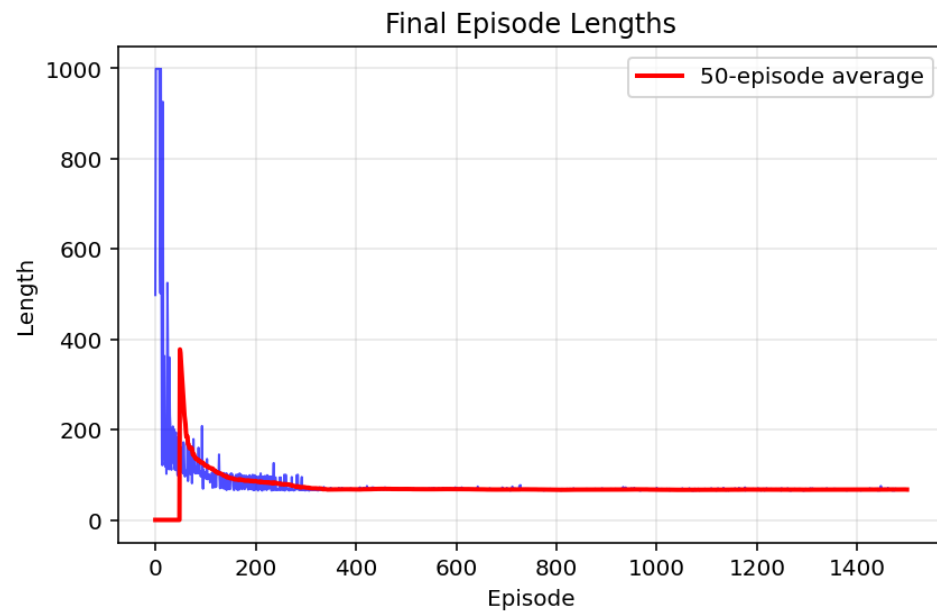
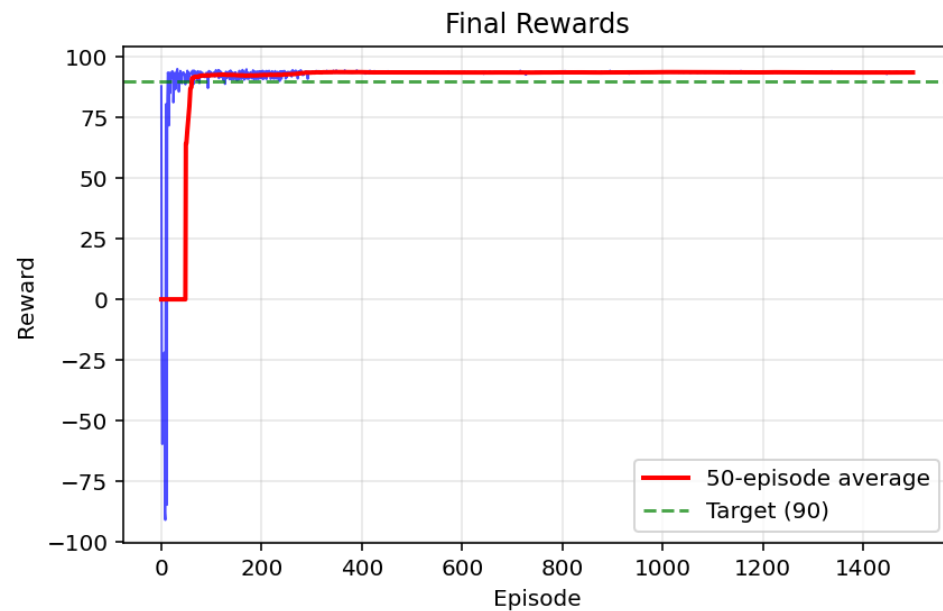
18:   end if
19: end for
20: end if
21: until convergence

```



# Training Result

---



# Reference

---

- ▶ M. Lapan, *Deep Reinforcement Learning Hands-On*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.
- ▶ A. Zai and B. Brown, *Deep Reinforcement Learning in Action*. Shelter Island, NY: Manning Publications, 2020.
- ▶ S. Ravichandiran, *Deep Reinforcement Learning with Python*, 2nd ed. Birmingham, UK: Packt Publishing, 2020.